

Images and sequences.

Structure meets learning.

Read a poster, follow a viewing history,
and understand how models reuse learned features.

🕒 20-25 min core reading · 20-30 min practice · 12 short sections

A movie poster has neighboring pixels. A viewing history has an order. A model trained on another image task may already contain useful visual features. This chapter explains how networks use those structures, and how to reuse learning without confusing a working calculation with a useful recommendation system.

WHAT YOU WILL UNDERSTAND

The main question: which parts of a network should share information, preserve order, or reuse previous learning?

The practical path: calculate a convolution, trace a recurrent state, and inspect which parameters change when a backbone is frozen.

Reading path: twelve short sections, three diagrams, three small interactive experiments, and a Python workbook. Gate equations and implementation details are optional.

1. Let the input structure guide the architecture

An **architecture** specifies how layers connect and information flows. The loss and optimizer still determine what is learned. An **inductive bias** is a built-in preference for certain kinds of patterns, such as local visual patterns or dependencies across time.

Movie input	Structure worth preserving	A possible model
Runtime, year, genre flags	A fixed set of named features	Linear model, tree, or dense network
Poster pixels	Nearby pixels form edges and shapes	Convolutional neural network (CNN)
Ordered viewing events	Earlier events can influence later context	Recurrent neural network (RNN) or another sequence model

These are useful starting points, not exclusive assignments. One-dimensional convolutions also process sequences; transformers can process images. A poster category is not a viewer's taste. A recommender may combine image features with catalog facts, viewing history, and an explicit ranking objective.

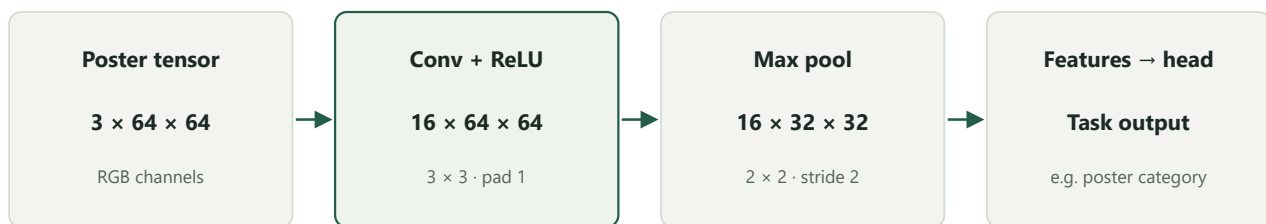
2. See how a convolution reads a local patch

A **kernel**, or filter, is a small array of weights. It slides across the image; at each position it multiplies matching values and sums them, then adds a bias. The resulting grid is a **feature map**. The same weights are reused at every position: **weight sharing**.

Our fictional grayscale poster has five identical rows $[1, 1, 0, 0, 0]$, where 1 is bright and 0 is dark. A vertical-contrast filter has three identical rows $[1, 0, -1]$. At the first position, each row contributes $1 \times 1 + 1 \times 0 + 0 \times (-1) = 1$. Three rows give **3**, with bias zero. Moving one pixel right gives 3 again; moving twice gives 0. The output is a 3×3 grid with rows $[3, 3, 0]$.

This operation does not flip the kernel; technically it is **cross-correlation**, the convention commonly called convolution in neural networks. Our filter is hand-set so its arithmetic is visible. In a trained CNN, backpropagation learns filters from the task loss; they are not all predefined edge detectors. See D2L's convolution explanation.

➤ A poster becomes a learned representation



Local connections + shared filters → fewer parameters

Batch dimension omitted. A filter spans every input channel.

Figure 1. A CNN reuses local filters across a poster. Activations and later layers form richer representations; pooling reduces spatial detail, and a task head maps features to an output. Shapes show one illustrative design.

3. Track channels, stride, padding, and pooling

An RGB image has three **channels**: red, green, and blue. One ordinary convolutional filter spans all input channels and sums their contributions into one output map. Sixteen filters produce sixteen output channels. Batch size is a separate dimension; one common layout is batch × channels × height × width. Other libraries use different axis orders. [D2L develops multi-channel convolution](#).

Stride is how far the kernel moves per step. **Padding** adds a border, often zeros, so the kernel can operate near image edges. For input height H , kernel height k , symmetric padding p on each side, and stride s , with dilation 1:

$$H_{\text{out}} = \left\lfloor \frac{H+2p-k}{s} \right\rfloor + 1.$$

Apply the same rule to width. Our 5×5 input with a 3×3 kernel gives 3×3 at stride 1 without padding, 5×5 with padding 1, and 3×3 with padding 1 and stride 2. The floor discards an incomplete final placement.

[Padding and stride details](#).

ReLU replaces negative values with zero. **Max pooling** keeps the largest value in each local window; average pooling keeps the mean. A 2×2 max pool with stride 2 reduces our ReLU output to the single value 3, dropping incomplete border windows here. **Global average pooling** instead averages every spatial position in each channel; our single channel becomes 2. Pooling has no learned weights in these forms and loses location information. [Pooling reference](#).

▼ Go deeper: count parameters, not output cells

For a standard convolution with groups = 1 and one bias per output channel, the parameter count is $(k_H k_W C_{\text{in}} + 1) C_{\text{out}}$. A 3×3 convolution from 3 channels to 16 needs $(3 \times 3 \times 3 + 1) \times 16 = 448$ parameters. Its output may contain thousands of values, but those are activations, not independent weights.

Flattening a 64×64 RGB image and connecting it to 128 dense units needs $(64 \times 64 \times 3 + 1) \times 128 = 1,572,992$ parameters. These layers produce different outputs, so this is a connectivity comparison, not a matched-quality benchmark. **Dilation** spaces kernel samples apart; with dilation d , replace k in the shape formula by the effective size $d(k - 1) + 1$.

4. Build representations from local to broad

A **receptive field** is the input region that can influence a particular activation. Two 3×3 , stride-1 convolutions without dilation have a theoretical 5×5 receptive field for an interior output. Later units can combine earlier patterns over a wider region. This makes structures such as outlines and arrangements available to the model; it does not guarantee that a particular unit has a human-readable meaning.

A **backbone** is the main feature-producing network. A **head** maps its representation to the task: for example, poster-category logits, bounding-box coordinates, or a vector used with viewer features. Classification, object detection, and pixel-level segmentation require different outputs and labels.

Weight sharing helps a filter respond to a pattern in different places. It does **not** guarantee that the whole model gives identical predictions after every shift, crop, rotation, or scale change. Borders, downsampling, and the head matter. Augmentations should preserve the label: cropping away the only relevant poster detail may not.

▼ Go deeper: receptive fields and residual connections

Track a layer's receptive-field size r and spacing j between neighboring features in input coordinates. Start with $r = 1, j = 1$. A layer with kernel k , dilation d , and stride s gives $r' = r + (k - 1)d$ and $j' = js$. Two 3×3 stride-1 layers give $r = 3$, then $r = 5$. A stride-2 pooling layer then changes both coverage and spacing. This describes possible connectivity, not how strongly each input actually contributes.

A **residual connection** adds an input to a learned transformation: $y = x + F(x)$ when shapes agree; a projection can align mismatched shapes. The shortcut provides a direct route for information and gradients. It does not eliminate the need for good training or guarantee that arbitrarily deep models work. See the original [ResNet paper](#).

5. Preserve the order of viewing events

Consider **Adventure** → **Adventure** → **Romance** and **Romance** → **Adventure** → **Adventure**. They contain the same genres but end differently. A count or mean cannot distinguish them; an ordered model can. Whether recency improves recommendations is an empirical question, not an automatic property of sequence modeling.

Represent each event by a vector: perhaps a movie embedding, watch duration, and elapsed time. An **embedding** is a learned numerical representation; our experiment instead uses the authored scalar codes **adventure** = +1 and **romance** = -1. These codes describe the watched genre, not whether the viewer liked it.

Sequence tasks differ. **Many-to-one** uses a history to produce one result, such as a next-item score. **Many-to-many** produces an output per position, such as predicting the next item after each prefix. Encoder-decoder models map an input sequence to an output sequence, possibly of another length. A next-event predictor may use only information available before that event.

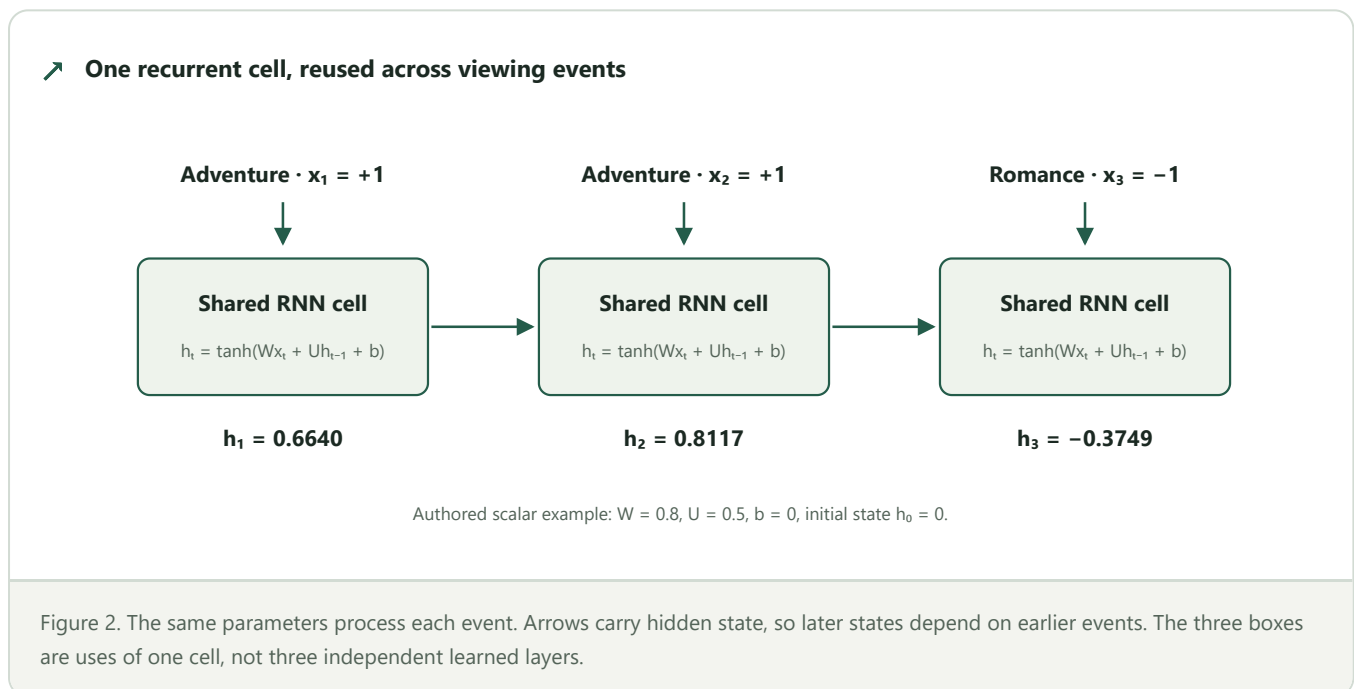
6. Trace a recurrent network through time

An RNN maintains a **hidden state**: a numerical summary updated after each event. A simple cell uses

$$h_t = \tanh(Wx_t + Uh_{t-1} + b).$$

W maps the current input, U maps the previous state, and b is a bias. The parameters are shared across time. A separate output head turns the state into task predictions. The state itself is not a probability. [RNN explanation](#).

Our scalar example starts at $h_0 = 0$ with $W = 0.8$, $U = 0.5$, $b = 0$. The original history produces states **0.6640** → **0.8117** → **-0.3749**. Reversing it produces **-0.6640** → **0.4366** → **0.7692**. Both input means are 0.3333. The different final states show order sensitivity, not measured recommendation quality.



During training, **backpropagation through time (BPTT)** applies the chain rule to the unrolled sequence and accumulates contributions to the shared parameters. Long chains can weaken or amplify gradients, linking back to Chapter 21's vanishing/exploding-gradient problem. **Truncated BPTT** cuts the gradient path at chosen boundaries; it may carry the numerical state forward while limiting how far credit can travel. [BPTT reference](#).

▼ Go deeper: padding, masks, and the future-information boundary

Batching variable-length histories often requires padding. A padding value is a placeholder, not another movie. At a padded position our mask keeps $h_t = h_{t-1}$. Merely setting $x_t = 0$ still computes $\tanh(Uh_{t-1} + b)$ and usually changes the state. Mask loss terms too when padded positions have no target. Packing sequences or selecting the last valid state are other implementation techniques.

Reset state between unrelated viewers unless you deliberately manage separate state for each. A **bidirectional RNN** reads both directions; it can be useful when the complete input is available, such as classifying a finished review. A prediction at time t must not access later events through a backward state, preprocessing, or the split. Histories and labels should be constructed at their actual prediction cutoffs.

7. Understand why LSTMs and GRUs add gates

A plain RNN repeatedly transforms its state. **Gates** are learned controls, usually sigmoid values between 0 and 1, that regulate information flow. They are computed from inputs and state, not manual switches.

An **LSTM** (long short-term memory network) maintains a cell state as well as a hidden state. A forget gate controls how much old cell content is retained, an input gate controls new content, and an output gate controls what is exposed through the hidden state. The additive cell path can make long-range learning easier. It does not provide perfect recall. [LSTM reference](#).

A **GRU** (gated recurrent unit) combines memory into a hidden state, using reset and update gates to control candidate computation and blending. It generally has fewer parameter groups than an LSTM of the same input and hidden size. Neither is universally better; compare accuracy, latency, and data requirements. [GRU reference](#).

▼ Go deeper: read a gate equation without memorizing it

A common LSTM formulation is $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ and $h_t = o_t \odot \tanh(c_t)$. Here f, i, o are forget, input, and output gates; $\tilde{c}$ is proposed new content; $\odot$ means elementwise multiplication. Each gate is a learned function of the current input and previous hidden state. If one coordinate has old memory 0.8, forget gate 0.9, input gate 0.2, and candidate -0.5 , its new cell value is $0.9(0.8) + 0.2(-0.5) = 0.62$ before the output gate is applied.

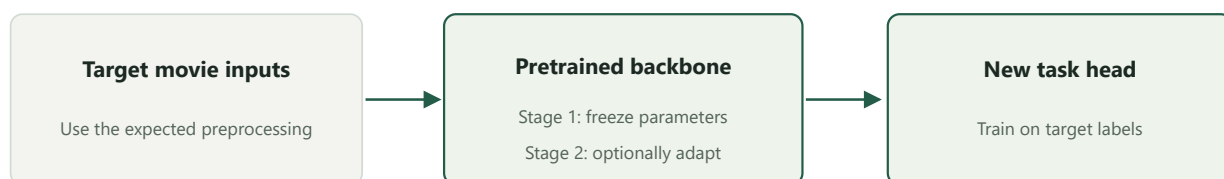
One GRU convention writes $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$, where z controls retention and a reset gate influences the candidate $\tilde{h}$. Some references reverse the update-gate convention, so read the equation before interpreting “open.” The workbook implements only a plain scalar RNN; it does not implement LSTM or GRU training.

8. Reuse a learned backbone through transfer learning

Transfer learning reuses knowledge learned from a source task or dataset for a target task. For posters, a backbone pretrained on broad images may supply useful features. Replace its original classification head with one matching your poster labels, or combine its features with viewer information for recommendation.

With **feature extraction**, freeze backbone parameters and train a new head. With **fine-tuning**, update some or all pretrained parameters on the target objective. With **training from scratch**, start without those learned weights. Keeping the backbone frozen does not mean that no learning occurs: the new head still learns. The PyTorch transfer-learning tutorial illustrates both reuse strategies.

↗ Reuse a backbone; choose where learning may happen



Validate the choice → freeze the policy → evaluate independently

The workbench demonstrates freezing with hand-set weights; it does not load this pretrained system.

Figure 3. A pretrained backbone supplies features to a new task head. First train the head with the backbone frozen; then optionally unfreeze selected layers and compare on validation. Freezing parameters and evaluation mode are separate controls.

Transfer may help with limited target labels, but mismatched source features can also hurt: **negative transfer**. A source model's accuracy or confidence does not establish target-domain accuracy. The feature-extractor contract includes expected image size, channel order, normalization, weights version, and output representation. Preserve that contract when serving predictions.

9. Adapt a model with a fair development process

Start with a simple baseline and a target split reflecting deployment: new movies, new viewers, or future events. Avoid near-duplicate posters or later viewing feedback crossing the relevant boundary. Train the new head first, then consider unfreezing later backbone blocks. Use validation to choose that policy and its learning rates; pretrained layers often benefit from smaller updates than a new head. Keep a separate final evaluation after selection. See [D2L on fine-tuning](#).

Freezing parameters, setting evaluation mode, and disabling gradient recording are different operations. For a strictly fixed backbone, prevent parameter updates **and** prevent changing state such as batch-normalization running statistics. Fine-tuning may intentionally use a different policy. If you unfreeze layers, make sure the optimizer includes them and that gradient recording is enabled through them.

▼ Go deeper: the fixed-backbone implementation boundary

This illustrative PyTorch pattern assumes a loaded backbone, a newly created head, correctly preprocessed batches, and a task-compatible loss. It is not executed by the standard-library workbook:

```

for parameter in backbone.parameters():
    parameter.requires_grad_(False)
backbone.eval() # also keep its dropout / running-stat behavior fixed
head.train()
optimizer = torch.optim.Adam(head.parameters(), lr=0.001)

for images, labels in target_train_loader:
    optimizer.zero_grad(set_to_none=True)
    with torch.no_grad():
        features = backbone(images)
    loss = loss_fn(head(features), labels)
    loss.backward()
    optimizer.step()

```

Here `torch` is imported separately, and the backbone must return features rather than its old class scores. Calling `train()` on a parent model can change child modes again, so reapply the intended backbone mode when needed. Unfreezing requires revisiting all three decisions: parameter flags, optimizer membership, and the no-gradient block. Cached features must be recomputed if the backbone or preprocessing changes.

10. Explore the calculations and practice

In **Poster filter**, select output cells to follow each multiply-and-sum, then change stride or padding. In **Viewing history**, reverse the events and disable the padding mask. In **Freeze or adapt**, compare exactly which parameters change after one gradient step.

Same building blocks. Different structure.

Slide a filter, carry a state, or choose which parameters may change. All fixtures and starting weights are hand-set; no pretrained model is loaded.

Poster filter: for the bright-left pattern and vertical filter, stride 1 and no padding, the raw output has three identical rows [3, 3, 0]. ReLU leaves it unchanged. A 2×2 max pool with stride 2 returns [3]; global average pooling returns 2.

Viewing order · $u = 0.5$	Input mean	Last real state
Adventure → Adventure → Romance	0.3333	-0.3749
Romance → Adventure → Adventure	0.3333	0.7692

Masking the two padding positions preserves the final real state. Without a mask, the original history ends at -0.0924 instead of -0.3749.

One-step update policy	Trainable	Loss before	Loss after
Frozen backbone	3 / 9	0.5351	0.5134
Adapt all parameters	9 / 9	0.5351	0.5043

The largest backbone change is 0 when frozen and 0.0199 when adapted. The head's first update is identical in both modes because it uses the same starting features. These are local arithmetic results, not a pretrained-model benchmark.

The third experiment uses a hand-set $2 \rightarrow 2 \rightarrow 1$ network, with six backbone parameters and three head parameters. It starts from the same weights for each policy. These weights were **not pretrained**. The example demonstrates freezing and adaptation mechanics, not the benefits of real transfer learning. There is no held-out dataset or quality benchmark in this chapter.

The four-file Python workbook includes `data.json`, `explore.py`, `test_explore.py`, and `README.md`. It requires Python 3.10+ and the standard library, with no model downloads or API calls.

```
py explore.py cnn --poster edge --padding 1 --stride 2
py explore.py sequence --history reversed
py explore.py sequence --no-mask
py explore.py transfer --mode frozen
py explore.py transfer --mode finetune
py -m unittest -v test_explore.py
```

Use `python` or `python3` if needed. Commands print JSON; they do not save a model. Invalid CLI choices exit with code 2. The report exposes intermediate states, features, and parameter updates so you can reproduce the reasoning.

Exercise 1: distinguish shape from parameter count

A 32×32 RGB image enters a 3×3 convolution with 8 output channels, padding 1, stride 2, dilation 1, and a bias per filter. What is the output shape, omitting batch size, and how many parameters are learned?

▼ Reveal the reasoning

Each spatial dimension is $\text{floor}((32 + 2 - 3) / 2) + 1 = 16$. The output is $8 \times 16 \times 16$ in channel-first layout.

Parameters = $(3 \times 3 \times 3 + 1) \times 8 = 224$. There are 2,048 output activations, but they reuse those 224 parameters.

Exercise 2: explain a padding bug

A real history ends at hidden state -0.3749 . Two zero-padding positions are processed with $h_t = \tanh(0.5h_{t-1})$. Why does the reported final state change, and what should a mask do?

▼ Reveal the reasoning

The recurrent transform is still applied even though the input is zero. The state becomes about -0.1853 and then -0.0924 . Padding is not a real observation; skipping both updates preserves -0.3749 . The changed value does not mean that the viewer watched two neutral movies.

Exercise 3: interpret the adaptation comparison

In the toy network, freezing gives loss $0.5351 \rightarrow 0.5134$; adapting all parameters gives $0.5351 \rightarrow 0.5043$ after one step. How many parameters can change in each case? Does the lower second loss establish better recommendations?

▼ Reveal the reasoning

Only the 3 head parameters may change when the backbone is frozen; all 9 may change when it is adapted. Both losses are measured on the same training example. Neither result establishes generalization, and the starting weights are hand-set rather than pretrained. A real comparison needs target data, tuned policies, and separate evaluation.

11. Check your understanding

01 Why can the same CNN filter detect a pattern at different image positions?

- A. Its weights are shared across spatial positions.
- B. It stores a separate full model for each pixel.
- C. Pooling guarantees perfect translation invariance.

Answer: A. One kernel slides across the image. Sharing reuses the same local detector; borders, stride, and later layers still affect how shifts change the result.

02 A 5×5 input uses a 3×3 filter, padding 1 per side, stride 2, and dilation 1. What is the output size?

- A. 5×5
- B. 3×3
- C. 2×2

Answer: B. Each dimension is $\text{floor}((5 + 2 - 3) / 2) + 1 = 3$. Padding and stride must both be included.

03 What does a negative hidden state in the viewing-history experiment mean?

- A. The viewer definitely dislikes the last movie.
- B. A negative probability was predicted.
- C. It is an internal signed representation, not a preference probability.

Answer: C. The scalar state is computed from authored genre codes and fixed weights. A trained output head and task objective would be needed to interpret it as a prediction.

04 What should happen at a masked padding position in this RNN?

- A. The state must become zero.
- B. The previous state should be preserved.
- C. The zero input should always be processed as another viewing event.

Answer: B. The padding position is not an observation. Merely inserting $x = 0$ still changes $\tanh(u h)$, so the mask skips that update.

05 A frozen backbone plus a trained head improves loss on one movie. What can you conclude?

- A. Transfer learning is proven better on unseen users.
- B. All backbone weights were updated.
- C. The head learned on this example; generalization still needs separate evaluation.

Answer: C. Freezing keeps backbone parameters unchanged. A training-loss decrease is local evidence, not a held-out result or a comparison of transfer strategies.

12. Connect architectures to language

You can now ask three concrete questions about a network: **which local patterns does it reuse, how does it represent order, and which learned parts can change?** Architecture supplies a way to represent patterns; the task definition, labels, optimization, and evaluation determine whether those patterns help.

The next chapter, **NLP, tokenization, and language modeling**, turns words into model inputs and explains the objective of predicting text. Attention and transformers follow in Chapter 24. Sequence models in this chapter provide the context for understanding what those architectures change.

▼ Quick-reference glossary

Term	Meaning
Kernel / feature map	Shared local weights / the grid of their responses
Channel / batch	A feature plane / a group of examples processed together
Stride / padding / dilation	Movement between windows / added border / spacing within a kernel
Pooling / receptive field	Spatial aggregation / input region that can influence a unit
Backbone / head	Feature-producing network / task-specific output mapping
Hidden state / BPTT	A recurrent representation / backpropagation through the unrolled sequence
Mask / gate	Exclusion of invalid positions / a learned control of information flow
LSTM / GRU	Two recurrent architectures with learned gates
Transfer / fine-tuning	Reuse of previous learning / adaptation of pretrained parameters
Negative transfer	Reuse that harms performance on the target task

Sources and further reading

Primary learning references are linked beside the relevant concepts: D2L's CNN and recurrent-network chapters, the ResNet paper, and the PyTorch transfer-learning tutorial. Poster patterns, viewing histories, arithmetic examples, diagrams, and workbook are original teaching material. The local experiments require no pretrained weights, framework installation, or external service.